

Language-Guided Planning and Replanning for Autonomous UAVs in Unknown Environments

Justin Turnau
Arizona State University
Tempe, USA
jturnau@asu.edu

Abstract—We present a language-guided planning and replanning framework for autonomous UAVs operating in unknown environments, in which a large language model (LLM) serves as both the initial mission planner and the dynamic replanner invoked at every skill boundary and event trigger over an accumulated context of execution history and discovered object observations. Plans are expressed as sequences of parameterized skill invocations from a fixed vocabulary, making them directly executable on a PX4/ROS2 flight stack without intermediate compilation. We evaluate three models across four offline experiments covering initial plan generation, isolated failure recovery, multi-stage replanning chains, and compound conflicting failures, with Gemini-Flash-2.5 achieving 100% replan decision accuracy on multi-stage chains and 75% mission success under compound failures, and demonstrate closed-loop end-to-end execution in a Gazebo simulation environment.

Index Terms—Autonomous UAVs, large language models, mission planning, replanning, PX4, ROS2, language-guided robotics, autonomous systems, human-robot interaction, adaptive planning

I. INTRODUCTION

Autonomous UAVs are increasingly deployed in scenarios that require flexible, goal-directed behavior in environments that are not known ahead of time: infrastructure inspection, search and rescue, and planetary exploration all demand a system that can translate high-level intent into executable action and recover when execution diverges from expectation. Classical planning approaches address this by precomputing plans over explicit world models, but such models are unavailable when the environment is unknown and object locations must be discovered during flight.

Large language models (LLMs) offer a complementary capability: they encode broad world knowledge and can reason over natural language mission specifications, structured execution histories, and failure signals to produce contextually appropriate plans. Recent work has demonstrated LLM-driven planning for ground robots [1], autonomous underwater vehicles [2], and UAVs [3]–[5]. However, these efforts variously address short-term subgoal planning, explicit replanning triggers, reactive safety decisions, semantic navigation over known maps, or one-shot multi-UAV command generation, leaving the integration of initial plan generation, runtime monitoring, and context-aware replanning over accumulated execution history for UAVs in unknown environments as an open problem.

We present a unified framework that addresses this gap. A single LLM serves both as the initial mission planner, generating a structured plan from a natural language specification before flight, and as the dynamic replanner, invoked at every skill boundary and event trigger to determine whether the remaining plan still satisfies mission intent and to produce a corrected tail if not. Plans are expressed as sequences of parameterized skill invocations drawn from a fixed vocabulary of five primitives, making them directly executable on a PX4/ROS2 flight stack and validatable against a compact set of domain constraints. The replanner operates over an accumulated execution context comprising the completed skill history and a structured manifest of discovered objects, enabling adaptation to new discoveries and cascading failures.

We make the following contributions:

- A language-guided planning and replanning framework for autonomous UAVs in unknown environments that unifies initial plan generation and dynamic plan revision under a single LLM interface
- Offline evaluation across four experiments, including multi-stage replanning chains and compound conflicting failures, comparing Gemini-Flash-2.5, Qwen-235B, and LLaMA-3.1-8B
- End-to-end deployment in a PX4/Gazebo simulation demonstrating closed-loop plan generation and dynamic replanning on a real flight stack

II. RELATED WORK

Planning from natural language for autonomous systems has received substantial attention in recent literature. We organize the most relevant prior work into three categories: general autonomous systems, ground robot navigation, and UAV-specific approaches.

General Autonomous Systems. The work most closely related to our framework is [2], which demonstrates planning and replanning for autonomous underwater vehicles (AUVs) using a status evaluator and environment monitor that trigger replanning at discrete stages based on sensor data, for example, initiating motion-control replanning when IMU readings indicate extreme acceleration. In contrast, our framework delegates this decision entirely to the LLM replanner, which determines both whether replanning is necessary and what must change, based on a mission status report and event signals.

Ground Robot Navigation. [1] dynamically generates high-level plans for robotic navigation by focusing on short-term subgoals rather than long-horizon planning, enabling effective operation in unknown environments. While sharing our interest in adaptive replanning, this work targets ground navigation rather than UAV mission execution.

UAV-Specific Approaches. Several works have explored LLM-driven control and planning for UAVs. [4] uses a planner enabled by an LLM to conduct task-driven navigation for UAV–UGV (unmanned ground vehicle) collaboration. [6] demonstrates natural language control of UAVs using the same Gazebo and PX4 simulation stack we employ, validating the feasibility of LLM-interpreted commands in this environment. [7] similarly uses a PX4-based platform but focuses on low-level control through an LLM–VLM pair to reach mission objectives. [8] addresses a complementary problem: real-time anomaly detection and low-level fallback selection via embedding-based fast reasoning. Our work differs by focusing on high-level mission planning and replanning through LLM reasoning over accumulated execution context and mission semantics. [3] is the closest related UAV planning work, demonstrating that LLMs can make reactive safety decisions within a fixed mission structure. Our work extends this to the harder problem of dynamically revising multi-step mission plans as the environment is discovered. [5] addresses task planning in multi-UAV systems from natural language commands, and [9] introduces a system for mapping natural language to multi-scale UAV operations using an LLM as a semantic parser.

III. PROBLEM FORMULATION

We consider the problem of language-guided mission planning and *replanning* for an autonomous UAV operating in an unknown and unstructured environment. The UAV is tasked with executing high-level mission objectives specified in natural language, without access to prior maps, object locations, or environment structure.

A. Environment and System Model

We model the environment as a partially observable domain. At time step t , the UAV receives observation $o_t \in \mathcal{O}$ from onboard perception and executes a high-level skill $k_t \in \mathcal{K}$. Rather than maintaining a full belief over latent states, the system constructs a structured *execution context*:

$$h_t = (\mathcal{K}_{1:t}, \mathcal{M}_t),$$

where:

- $\mathcal{K}_{1:t} = [k_1, k_2, \dots, k_t]$ is the sequence of executed skills
- $\mathcal{M}_t$ is a memory of observed entities (e.g., cylinders), where each entry stores attributes such as relative position to the UAV at detection time and the UAV pose at detection

This representation captures both behavioral history and structured observations accumulated during execution.

B. Language-Guided Planning

With the goal of completing the user-specified mission, the system first receives a natural language mission specification:

$$g \in \mathcal{G}.$$

An LLM-based planner generates an initial structured plan:

$$P_0 = LLM(g, \text{PLANNING_PROMPT}),$$

where $P_0 = [k_1, k_2, \dots, k_m]$ denotes a sequence of parameterized skills, and `PLANNING_PROMPT` encodes the skill vocabulary, ordering constraints, and output format. At this stage, the execution context is empty ($h_0 = \emptyset$), as no skills have been executed and no objects have been observed; the LLM therefore plans under full environmental uncertainty.

C. Execution Under Uncertainty

Execution proceeds sequentially over skills. Due to partial observability, unknown environment structure, and other uncertainties, discrepancies may arise between expected and actual outcomes. We define execution signals:

$$\sigma_t = f(h_t, P_t),$$

which capture deviations using the structured execution context. These include:

- **Explicit:** direct failures (e.g., target not detected, sensor error)
- **Implicit:** inconsistencies between expected and observed progress (e.g., mapping without prior approach)
- **Historical:** signals derived from accumulated execution context (e.g., repeated unsuccessful searches)

D. LLM-Based Replanning

At decision points, the LLM determines whether to continue execution or replan:

$$d_t = LLM(g, h_t, P_t, \sigma_t),$$

where $d_t \in \{\text{continue}, \text{replan}\}$. If the LLM determines that a replan is necessary, a new plan is generated:

$$P_{t+1} = LLM(g, h_t, P_t, \sigma_t).$$

Both the decision to replan and the generation of updated plans are conditioned on the executed skill history and accumulated observation memory, enabling context-aware adaptation under uncertainty.

E. Objective

The objective is to execute a plan P_t or sequence of continually adapting plans $\{P_t\}$ that maximizes mission success while maintaining robustness to uncertainty and execution failures. This requires:

- 1) Generating valid and constraint-satisfying plans
- 2) Accurately determining when replanning is necessary
- 3) Producing updated plans that leverage accumulated skill and observation history

IV. METHODOLOGY

The framework takes a natural language mission specification as input and produces a sequence of parameterized skill invocations (plan) that is executed on the UAV. The system operates in two phases: initial plan generation before flight, and dynamic replanning triggered during execution.

A. Skill Vocabulary and Plan Representation

Rather than generating abstract state machines or symbolic programs, the LLM produces a structured plan as a JSON array of parameterized skill invocations drawn from a fixed vocabulary. Each step takes the form:

```
{"state": <skill_name>, "args": {...}, "repeat": <int>}
```

The skill vocabulary consists of five primitives: `takeoff`, `search`, `approach`, `map`, and `return_home`. Skills are parameterized, for instance, `search` accepts a pattern (`yaw_scan` or `lawnmower`), `approach` accepts a `cylinder_id` and `standoff_distance`, and `map` accepts a mode (`orbit` or `vertical_map`), `standoff` distance, and optional altitude. The `repeat` field allows a skill to be executed N times before advancing, enabling multi-pass mapping without duplicating plan steps.

This representation is deliberately constrained. The LLM cannot invoke arbitrary behaviors, it can only compose and parameterize skills from the fixed set. This makes plans directly executable by the flight stack without any intermediate compilation step, and it makes validity checking tractable: a plan is valid if and only if it satisfies a small set of structural and domain constraints (e.g., `takeoff` must be the first step, `approach(cylinder_id=X)` must immediately precede `map(cylinder_id=X)`, and `standoff` distances must satisfy mode-specific bounds).

B. Initial Plan Generation

At mission start, the LLM receives a natural language mission string g and a system prompt describing the skill vocabulary, argument types, ordering constraints, and output format. It generates an initial plan P_0 as a JSON array:

$$P_0 = \text{LLM}(g, \text{PLANNING_PROMPT})$$

No prior map or object locations are provided, meaning the LLM plans under full environmental uncertainty, relying on `search` steps to discover objects during execution. A lightweight validator checks P_0 for structural correctness and constraint satisfaction; if the plan is invalid, the LLM is queried (up to three attempts) with the specific validation errors appended to the prompt.

C. Execution Context

During flight, the system maintains an execution context h_t that accumulates information across skills:

$$h_t = (\mathcal{K}_{1:t}, \mathcal{M}_t)$$

where $\mathcal{K}_{1:t}$ is the ordered history of successfully completed skills and $\mathcal{M}_t$ is a structured memory of detected objects. Each entry in $\mathcal{M}_t$ stores the object's identifier, its estimated position in the world frame at detection time, and the depth at detection. This context grows monotonically throughout the mission and is passed in full to the replanner at each decision point. Note the LLM has no implicit memory between calls.

D. Dynamic Replanning

After each skill completes (or fails), the system evaluates whether the remaining plan still satisfies the mission intent given the current execution state. This evaluation is performed by querying the LLM with a structured prompt comprising four components: (1) the original mission string g ; (2) the completed step history $\mathcal{K}_{1:t}$ with semantic annotations distinguishing successful completions from aborted steps; (3) the discovered object manifest $\mathcal{M}_t$ with position data; and (4) a failure context σ_t describing any anomaly that occurred during the most recent skill.

The replanner produces one of two outputs: the literal token `NOMINAL` if the remaining plan is already sufficient, or a corrected JSON tail plan covering all steps from the current position to `return_home`, followed by a one-sentence justification. Formally:

$$d_t = \text{LLM}(g, h_t, P_t, \sigma_t), \quad d_t \in \{\text{NOMINAL}, \text{REPLAN}\}$$

$$P_{t+1} = \begin{cases} P_t & \text{if } d_t = \text{NOMINAL} \\ \text{LLM}(g, h_t, P_t, \sigma_t) & \text{if } d_t = \text{REPLAN} \end{cases}$$

The replanner operates only on the tail of the current plan, completed steps are fixed history and cannot be revised. This constrains the output space and focuses the LLM's reasoning on what remains to be done, rather than re-deriving the full plan from scratch. As with initial planning, produced tails are validated against domain constraints, and the LLM is re-queried with error feedback on invalid outputs.

Failure signals σ_t are classified into three types. **Explicit** signals arise from direct skill failures such as sensor errors or detection failures. **Implicit** signals capture inconsistencies between expected and observed progress, such as a `map` step being reached without a preceding `approach` having established valid standoff geometry. **Historical** signals emerge from patterns in h_t , such as repeated failed searches over the same region.

The replanning loop runs at every skill boundary regardless of whether a failure occurred. This allows the system to adapt to new object discoveries and mission drift without requiring a separate anomaly detector to trigger replanning.

V. EXPERIMENTS

We evaluate the proposed framework across three offline experiments measuring LLM planning and replanning quality, plus a demonstration of end-to-end execution in a physics-based simulator. All offline experiments use the same initial plan or replan prompt structure respectively as described in Section IV and are evaluated across three models: Gemini-Flash-2.5, Qwen-235B, and LLaMA-3.1-8B.

A. Experiment 1: Initial Plan Generation Quality

We assess how reliably each LLM generates structurally valid and semantically correct mission plans from natural language. We constructed a suite of 15 missions spanning three difficulty levels: L1 (simple, no target interaction), L2 (single target), and L3 (multi-target or complex). For each mission, the LLM was queried once with zero temperature.

Plans are evaluated on validity rate (proportion of missions yielding a structurally executable plan), tail match rate (fraction of valid plans whose step-by-step content exactly matches a reference plan on state, arguments, and repeat counts), and mission success rate (proportion of *all* missions producing both a valid and reference-matching plan).

Results are summarized in Table II and broken down by difficulty level in Fig. 1. Gemini-Flash-2.5 and Qwen-235B both achieved 100% validity across all 15 missions; Qwen-235B led on the primary metric with 86.7% mission success while Gemini-Flash-2.5 reached 60.0%, indicating its valid plans more frequently diverged from the expected plan. LLaMA-3.1-8B, only achieving a validity rate of 73.3% and a mission success rate of 33.3%, performed significantly worse on the more complex L2 and L3 missions.

TABLE I
INITIAL PLAN GENERATION PERFORMANCE SUMMARY OVER 15 MISSIONS.

Model	Valid (%)	Tail Match (%)	Mission Success (%)
Gemini-Flash-2.5	100.0	60.0	60.0
Qwen-235B	100.0	86.7	86.7
LLaMA-3.1-8B	73.3	45.5	33.3

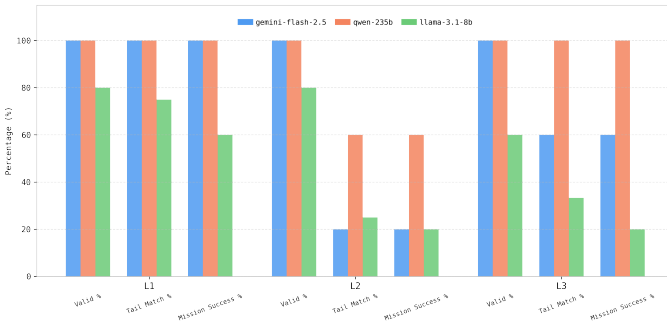


Fig. 1. Initial planning performance per-level (L1-L3) across models. Valid % measures the proportion of missions for which a model generated a structurally executable plan (i.e., satisfies ordering and state name constraints).

B. Experiment 2: Replanning Under Isolated Failure Events

We evaluate the replanner’s ability to diagnose and recover from a single failure signal at a given mission checkpoint. Scenarios are presented as static snapshots of the world state comprising completed steps, remaining steps, a cylinder manifest, and an optional failure context, injected directly into the replanner without executing in simulation. This design isolates replanning reasoning from execution variability.

Twelve scenarios are organized into three groups by signal type. **Explicit** signals provide raw sensor or perception events

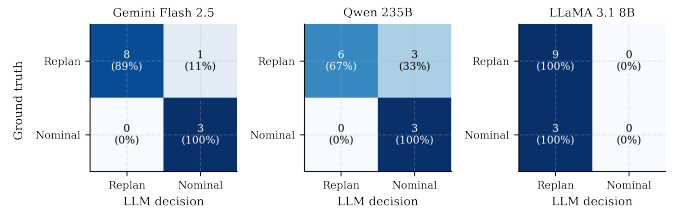


Fig. 2. **Replanning Under Isolated Failure Events** confusion matrices for each model showing the LLM decision to replan against ground-truth replanning requirements. Each matrix reports True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), along with their corresponding percentages over all evaluated missions.

(e.g., target tracking loss, battery telemetry). **Implicit** signals require the replanner to compare the cylinder manifest against the remaining plan without any textual hint. **Historical** signals require reasoning over the completed-step record alone (e.g., detecting that fewer mapping passes were completed than the mission required).

Each scenario carries a ground-truth label of NOMINAL or REPLAN, with REPLAN scenarios additionally specifying a reference tail. Performance is measured via mission success percentage, requiring both a correct decision and, for REPLAN scenarios, an exact step-wise tail match, alongside replan decision accuracy (binary NOMINAL/REPLAN only) and false positive and false negative rates.

Results are shown in Table II and confusion matrices in Fig. 2, with per-signal-type accuracy in Fig. 3. Gemini-Flash-2.5 achieved 83.3% behavior accuracy and 91.7% decision accuracy with zero false positives. Qwen-235B and LLaMA-3.1-8B both reached 75.0% decision accuracy but diverged on mission success (58.3% vs. 33.3%). The difference is clearly observed in Fig. 2, as Qwen-235B failed to identify three cases where replanning was necessary and LLaMA-3.1-8B attempted to replan for *every* mission. Lastly, mission success collapsed to zero on historical signals for Qwen-235B and LLaMA-3.1-8B, suggesting further improvements to this framework may be necessary to improve historical reasoning.

TABLE II
REPLANNING UNDER ISOLATED FAILURE EVENTS PERFORMANCE SUMMARY OVER 12 MISSIONS.

Model	Tail Match (%)	Mission Success (%)	Replan Decision Acc. (%)
Gemini-Flash-2.5	77.8	83.3	91.7
Qwen-235B	44.4	58.3	75.0
LLaMA-3.1-8B	44.4	33.3	75.0

C. Experiment 3: Multi-Stage Replanning Chains

We evaluate whether the replanner can be invoked *repeatedly* across an unfolding mission without compounding errors, losing execution state, or producing hallucinated plan changes. Eleven chain scenarios span four complexity tiers: simple (1 cylinder, 1 failure), medium (2 cylinders, 2–3 sequential failures), complex (3 cylinders, heterogeneous failure modes and mission types), and long (no-failure regression checks).

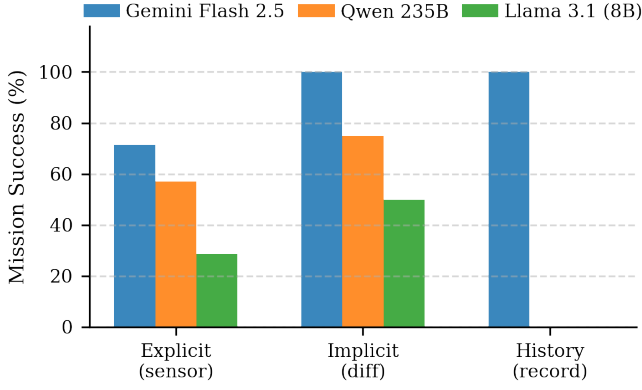


Fig. 3. **Correct recovery performance across signal types for simple missions.** Signal types include explicit sensor failures, implicit deviations, and historical context-based signals. Results show how effectively each model replans depending on the type of information available during failure recovery.

Each scenario is defined as a sequence of checkpoints. Between checkpoints, world state is updated via a structured event model: `AdvanceSteps` moves steps to history upon successful completion; `MarkCurrentStepFailed` removes the aborted step from the remaining plan without adding it to history, exposing a gap the replanner must detect from the manifest alone; `InjectCylinder` adds a newly discovered object; and `InjectFailure` updates the failure context. This distinction between successful completion and silent failure is critical: without it, failures become invisible to the replanner and every checkpoint produces a spurious NOMINAL.

Each checkpoint carries a reference tail, that is the exact correct step sequence the replanner should produce, compared via step-wise matching on state, mode, cylinder_id, standoff_distance, and repeat. We continue with **mission success**, **tail match**, and **replan decision accuracy** metrics as described in subsections V-B and V-C. However, when evaluating multi-stage replanning chains, we now calculate mission success as every checkpoint in the mission scenario succeeding. Furthermore, we now include **behavior accuracy** to measure the fine-grained behavior models, that is at every checkpoint whether they made the correct replan decision and requiring an exact tail match when replanning was necessary.

Results are presented in Table III and confusion matrices in Fig. 4. Gemini-Flash-2.5 achieved 96.3% behavior accuracy, 92.9% tail match, and a 100% decision accuracy with zero false positives or false negatives across all 11 chains. Qwen-235B achieved 92.6% behavior accuracy with a single false negative (7.1% FNR) but no false positives. Similar to its performance on isolated failure events, LLaMA-3.1-8B produced a 100% false positive rate on multi-stage chains, meaning it replanned at every checkpoint regardless of whether the plan was already correct. This resulting in an 18.2% behavior accuracy driven both by it always choosing to replan and its poor ability to replan correctly when required.

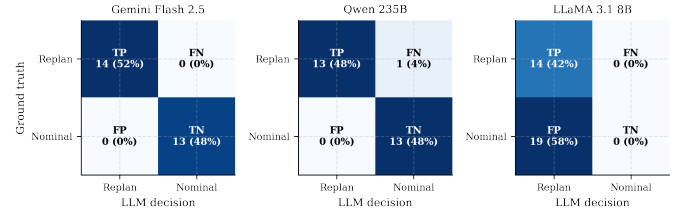


Fig. 4. **Multi-stage mission replanning** confusion matrices for each model showing the LLM decision to replan against ground-truth replanning requirements. Each matrix reports True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), along with their corresponding percentages over all evaluated missions.

D. Experiment 4: Replanning Under Compound Failures

The final experiment evaluates whether LLMs can correctly prioritize among multiple simultaneous and potentially conflicting failure signals, with no explicit priority rules provided in the prompt. This tests the implicit domain knowledge hypothesis: that models encode safety and mission hierarchies that generalize without rule enumeration.

Twelve scenarios are organized into four categories. **Safety vs. mission** scenarios present a critical battery or instability event alongside a mission-continuation signal; the correct response is immediate return regardless of remaining mission coverage. **Hard vs. soft** scenarios pair an unrecoverable constraint (physically blocked target, motor fault) with a soft warning (sensor drift, wind within tolerance) that should not independently alter the plan. **Conflict and adapt** scenarios present two signals that pull in opposite directions, requiring a principled partial-mission trade-off (e.g., mapping a nearby cylinder while skipping an energy-prohibitive distant one). **Recovery failure** scenarios present a second failure arriving on a plan that is already a recovery sequence, requiring the replanner to escalate rather than retry.

The same evaluation framework as Experiment 3 is applied, using reference tails and mission success percentage. Results are summarized in Table IV. Gemini-Flash-2.5 achieved 75.0% mission success percentage with an 8.3% false negative rate and no false positives. Qwen-235B achieved 41.7% mission success percentage with a higher false negative rate (16.7%) and lower accompanying decision accuracy than the Gemini-Flash-2.5 model. LLaMA-3.1-8B achieved 0.0% mission success percentage despite 100% decision accuracy, indicating that it consistently replanned, as NOMINAL cases were absent in the mission set, but produced tails that never matched the reference.

E. Practical Deployment

Beyond offline evaluation, the full system, LLM planner, FSM executor, skill library, and perception pipeline, is deployed and operational in a Gazebo simulation environment using PX4 and ROS2. This demonstrates closed-loop target search, approach, and mapping without prior environmental knowledge. A demonstration video of the system executing a simple language-guided mission in simulation is available

TABLE III
REPLANNING PERFORMANCE OVER 11 MULTI-STAGE CHAINED MISSIONS SPANNING L1–L3 COMPLEXITY AND TWO MISSIONS WITH ARE ADDED WITH NO FAILURE POINTS TO IDENTIFY FALSE POSITIVES.

TABLE IV
REPLANNING PERFORMANCE UNDER COMPOUND (SIMULTANEOUS) EVENTS ACROSS 12 SCENARIOS SPANNING FOUR CATEGORIES: *safety_vs_mission*, *hard_vs_soft*, *conflict_and_adapt*, AND *recovery_failure*. EACH SCENARIO PRESENTS 2+ CONCURRENT FAILURE SIGNALS WITHOUT EXPLICIT PRIORITY RULES, REQUIRING IMPLICIT CONFLICT RESOLUTION.

here: demo video. While thorough quantitative evaluation in simulation is reserved for future work, the demonstration confirms end-to-end plan generation, skill dispatch, and dynamic replanning on real PX4 flight stack outputs.

APPENDIX

A. Planning Prompt

The following system prompt is passed to the LLM for *initial* mission planning. Given a natural-language mission description by the user, the model is expected to return a valid JSON skill-sequence.

- [1] A. Rajvanshi, K. Sikka, X. Lin, B. Lee, H.-P. Chiu, and A. Velasquez, "Saynav: Grounding large language models for dynamic planning to navigation in new environments," in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 34, 2024, pp. 464–474.
- [2] R. Yang, F. Zhang, and M. Hou, "Oceanplan: Hierarchical planning and replanning for natural language auv piloting in large-scale unexplored ocean environments," in *Proceedings of the 18th International Conference on Underwater Networks & Systems*, ser. WUWNET '24. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3699432.3699496>
- [3] A. Tagliabue, K. Kondo, T. Zhao, M. Peterson, C. T. Tewari, and J. P. How, "Real: Resilience and adaptation using large language models on autonomous aerial robots," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 2024, pp. 1539–1546.
- [4] F. Cladera, Z. Ravichandran, J. Hughes, V. Murali, C. Nieto-Granda, M. Ani Hsieh, G. J. Pappas, C. J. Taylor, and V. Kumar, "Air-ground collaboration for language-specified missions in unknown environments," *IEEE Transactions on Field Robotics*, vol. 2, pp. 626–642, 2025.

You are a UAV mission planner.

Given a natural language mission description, produce a JSON execution plan as a sequence of parameterised skill invocations chosen from the vocabulary below. The UAV operates in an unknown environment - objects are discovered only through onboard perception during flight, not known in advance.

=== AVAILABLE SKILLS ===

Each plan step has the shape:

```
{ "state": <skill_name>, "args": { ... }, "repeat": <int, default 1> }
```

"repeat": N runs that skill to completion N times before advancing.

SKILL	ARGS	NOTES
takeoff	altitude: float (m)	ALWAYS the first step. Infer a reasonable altitude from context if not stated: confined space -> 2-3 m open area -> 5-8 m
search	pattern: "yaw_scan" "lawnmower"	Discovers objects via onboard perception during flight. yaw_scan - rotate in place; good for small areas or quick sweeps. lawnmower - systematic grid coverage; use when thorough search of a large area is

```
needed.

approach      cylinder_id: int | "all"
               standoff_distance: float (m, min 5.0)

               Fly to standoff_distance from
               the specified target(s).
               MUST appear before map for the
               same cylinder_id.

map           cylinder_id: int | "all"
               standoff_distance: float (m)
               mode: "orbit"
                   | "vertical_map"

               Two mapping modes available:

               ORBIT MODE (mode: "orbit"):
               Orbit the target once per
               invocation. standoff >= 5.0.
               Use repeat: N to orbit N
               times. approach for the same
               cylinder_id MUST precede.

               VERTICAL MAP MODE:
               Performs a systematic vertical
               sweep around the cylinder in
               four 90-degree arc segments.
               standoff 3.0-7.0 m.
               Use repeat: N for N full laps.
               approach for the same
               cylinder_id MUST precede.

               min_altitude_m: float
               (m, >= 2.0)

               Only for vertical_map mode.
               Default 2.0.

return_home (no args)

               Always the last step.

=== CONSTRAINTS ===

1. First step MUST be takeoff. Last step must be return_home.
2. approach(cylinder_id=X) MUST appear immediately before
   map(cylinder_id=X).
3. Do not include approach or map unless the mission explicitly
   involves object interaction.
4. Do not add steps that are not implied by the mission description.
5. For mode "orbit": standoff_distance must be >= 5.0 m for both
   approach and map.
6. For mode "vertical_map": standoff_distance must be between 3.0 m
   and 7.0 m.
7. After search, assume objects may be found. The automatic replanner
   will adjust the plan based on what is actually discovered.

=== OUTPUT ===

Respond with a JSON array only. No explanation, no markdown, no
backticks.
```

B. Replanning Prompt

The replanning prompt is constructed dynamically at run-time by `build_replan_system_prompt()`. The template below shows the full structure with placeholder fields rendered in `<angle brackets>`. All context is carried in the system prompt.

```
=== SECTION 1 - BACKGROUND ===

You are a mid-flight autonomous UAV mission replanner.

After each skill completes (or fails), you receive a status snapshot of
the ongoing mission. Your role is to review the remaining plan and decide
whether it still satisfies the user's original intent given what has been
discovered or has gone wrong so far. You may revise, extend, or trim the
remaining steps as needed. You do NOT re-plan from scratch - only the
tail (steps yet to execute) is yours to change. Completed steps are
fixed history.
Do NOT make changes unless they are ABSOLUTELY NECESSARY to complete the
intended mission.

=== SECTION 2 - USER MISSION ===

"<mission_intent>"

This is the exact mission the user requested. Use it as the ground truth
for what "success" means.

=== SECTION 3 - AVAILABLE SKILLS ===

Only the following skills may appear in a revised tail plan.
takeoff and search are already complete - do NOT include them.

SKILL      ARGS      NOTES
-----
approach    cylinder_id: int
               standoff_distance: float (m)

               Fly to standoff_distance
               from a specific cylinder.
               MUST immediately precede
               the map step for the same
               cylinder_id.
               orbit: standoff >= 5.0 m
               vertical_map: 3.0-7.0 m

map         cylinder_id: int
               standoff_distance: float (m)
               mode: "orbit"
                   | "vertical_map"

               Two modes:

               ORBIT: one full orbit per
               invocation (standoff >= 5.0).
               Use repeat: N to orbit N
               times.
               VERTICAL_MAP: systematic
               vertical sweep (3.0-7.0 m).

               min_altitude_m: float (>= 2.0)

               Must be the last step.

return_home (no args)

[SECTION 3b - FAILURE / ANOMALY REPORT]
```

```
[Included only when a failure_context is present:]

=== SECTION 3b - FAILURE / ANOMALY REPORT ===

The following event occurred during execution that may require you to
revise the remaining plan:

<failure_context>

Consider whether the remaining plan still makes sense given this event.

[End of optional section 3b]

Required pattern for each cylinder you intend to map:
{"state": "approach",
 "args": {"standoff_distance": D, "cylinder_id": N}},
{"state": "map",
 "args": {"mode": "orbit", "standoff_distance": D, "cylinder_id": N},
 "repeat": K}

Hard constraints:
- orbit: standoff_distance >= 5.0 for both approach and map
- vertical_map: 3.0 <= standoff_distance <= 7.0 for both
- min_altitude_m >= 2.0 (vertical_map only)
- approach(cylinder_id=X) MUST immediately precede map(cylinder_id=X)
- "repeat" is a TOP-LEVEL step field, never inside args
- Last step MUST be return_home
- Do NOT add approach or map for already-mapped cylinders

=== SECTION 4 - MISSION STATUS REPORT ===

COMPLETED STEPS (fixed history):
<completed_steps>

MAPPING PROGRESS:
    Only completed "map" steps count as mapping passes.
<mapping_progress>

<unmapped_note>

REMAINING PLAN (scheduled for execution):
<remaining_steps>

DISCOVERED OBJECTS:
<cylinder_manifest>

=== SECTION 5 - OUTPUT DIRECTIVE ===

Do NOT repeat, echo, or summarise any part of this prompt in your
response.

Carefully compare the remaining plan against the user mission.
If the remaining plan already fulfils the user mission, respond with
exactly:

NOMINAL

ONLY if the remaining plan MUST be adjusted, output:
1. A corrected JSON array of the complete revised tail (all steps from
now until return_home). Every element must be a complete JSON
object.
2. Immediately after the JSON, on a new line starting with "REASON:",
a single concise sentence explaining what was wrong and why your
revision is necessary.

Do not include any other explanation, markdown, or backticks.
```

1) Runtime-substituted fields: Table V describes each placeholder field injected in the replanning prompt at runtime.

The following tables enumerate every scenario used across the four offline experiments, together with the exact mission text and ground-truth expected outcome. “Expected outcome” is the exact response the replanner or planner must produce to be counted correct under the primary metric.

Skill-sequence notation: **TKO**(h) = takeoff to h m; **YS** = search(yaw_scan); **LM** = search(lawnmower); **A**(n, d) = approach cylinder n at standoff d m (default 5 m); **M**(n) = map orbit cylinder n (standoff 5 m); **VM**(n, d) = map vertical_map cylinder n at standoff d m; $\times k$ = repeat k times; **RH** = return_home; **NOM** = NOMINAL (no change to remaining plan).

Experiment 1 — Initial Plan Generation (15 Missions)

Experiment 2 — Replanning Under Isolated Failures (12 Scenarios)

Experiment 3 — Multi-Stage Replanning Chains (11 Chains)

Experiment 4 — Replanning Under Compound Failures (12 Scenarios)

Field	Description
<mission_intent>	Verbatim natural-language mission string supplied by the user.
<completed_steps>	Formatted list of skill invocations already executed.
<mapping_progress>	Per-cylinder count of completed mapping passes; sourced by scanning map entries in the completed-step history.
<unmapped_note>	One-line summary listing cylinder IDs not yet mapped, or a confirmation that all cylinders have been mapped.
<remaining_steps>	Formatted list of skill steps scheduled for future execution.
<cylinder_manifest>	JSON array of discovered cylinder objects, each with <code>cylinder_id</code> , <code>world_ned_x_m</code> , <code>world_ned_y_m</code> , and <code>depth_at_detection_m</code> .
<failure_context>	Free-text description of any anomaly or failure event; this entire section (3b) is omitted when no failure has occurred.

TABLE V
RUNTIME-SUBSTITUTED FIELDS IN THE REPLANNING SYSTEM PROMPT.

TABLE VI

EXPERIMENT 1: INITIAL PLAN GENERATION MISSION SUITE SPANNING THREE DIFFICULTY LEVELS. LEVEL 1: NO TARGET INTERACTION. LEVEL 2: SINGLE TARGET. LEVEL 3: MULTI-TARGET OR COMPLEX. THE EXPECTED PLAN IS THE CANONICAL REFERENCE TAIL; ANY POSITIVE ALTITUDE IS ACCEPTED WHERE ALTITUDE IS UNSPECIFIED (SHOWN AS TKO(*)).

ID	Lvl	Exact Mission Text	Expected Plan
L1-01	1	"Take off and return home."	TKO(*) → RH
L1-02	1	"Take off to 5 meters, do a quick scan of the area, then come back."	TKO(5 m) → YS → RH
L1-03	1	"Perform a thorough search of the field and return to base."	TKO(*) → LM → RH
L1-04	1	"Fly up to 3 meters, sweep the area for anything interesting, land at home."	TKO(3 m) → YS → RH
L1-05	1	"Conduct a systematic grid search of the exploration zone and return when done."	TKO(*) → LM → RH
L2-01	2	"Search for cylinders and map the first one you find, then come home."	TKO(*) → YS → A(1) → M(1) → RH
L2-02	2	"Find cylinders and do a vertical mapping sweep of each one."	TKO(*) → Search → A(all) → VM(all, 5 m) → RH
L2-03	2	"Scan for objects. If you find any, orbit the first one twice, then return."	TKO(*) → YS → A(1) → M(1)×2 → RH
L2-04	2	"Take off to 6 meters, do a panoramic scan, approach any cylinder at 5 meters and orbit it once."	TKO(6 m) → YS → A(all, 5 m) → M(all, 5 m) → RH
L2-05	2	"Search the area thoroughly, then do two full vertical mapping laps on each cylinder you find."	TKO(*) → LM → A(all) → VM(all, 5 m)×2 → RH
L3-01	3	"Search for cylinders and map every one you find with a single orbit, then return."	TKO(*) → Search → A(all) → M(all) → RH
L3-02	3	"Find any cylinders and map each one 2 times, then come home."	TKO(*) → Search → A(all) → M(all)×2 → RH
L3-03	3	"Perform a comprehensive survey: take off to 8 meters, do a lawnmower search, then do a vertical mapping sweep on every cylinder found, and return to launch."	TKO(8 m) → LM → A(all) → VM(all, 5 m) → RH
L3-04	3	"Take off, scan quickly, then orbit any cylinders you find three times each to build a detailed map. Come home afterwards."	TKO(*) → YS → A(all) → M(all)×3 → RH
L3-05	3	"Survey the area using a grid search pattern. For each cylinder discovered, do two vertical mapping laps at a close 4 meter standoff. Return home when complete."	TKO(*) → LM → A(all, 4 m) → VM(all, 4 m)×2 → RH

TABLE VII
EXPERIMENT 2: REPLANNING UNDER ISOLATED FAILURE EVENTS. GROUP A: EXPLICIT SENSOR/PERCEPTION EVENTS. GROUP B: IMPLICIT WORLD-STATE REASONING (NO FAILURE CONTEXT; REPLANNER MUST COMPARE MANIFEST TO PLAN). GROUP C: RAW BATTERY TELEMETRY ONLY. NOM = NOMINAL (NO CHANGE REQUIRED). TAIL NOTATION AS DEFINED ABOVE.

ID	Grp / Signal	Exact Mission Text	Failure Context Provided to Replanner	Expected Outcome
A1	A / Explicit	“Search for cylinders and map every one you find, then return.”	“Perception lost track of cylinder 2 during approach. No confirmed detection in the last 10 seconds.”	REPLAN $\rightarrow$ A(2) $\rightarrow$ M(2) $\rightarrow$ RH
A2	A / History	“Find cylinders and do 2 vertical mapping laps of each one you find.”	None — completion record shows only 1 of 2 required laps done; remaining plan skips directly to RH.	REPLAN $\rightarrow$ A(1, 5 m) $\rightarrow$ VM(1, 5 m) $\rightarrow$ RH
A3	A / Explicit	“Find cylinders and orbit each one twice, then come home.”	“Setpoint timeout while orbiting cylinder 2. The drone is holding position at the last commanded waypoint.”	REPLAN $\rightarrow$ A(2) $\rightarrow$ M(2) $\times$ 2 $\rightarrow$ RH
A4	A / Explicit	“Search for cylinders and do a vertical map of each one.”	“Vertical map skill failed to initialize: cylinder 1 centroid is outside the camera frame. Drone yaw may have drifted.”	REPLAN $\rightarrow$ A(1, 5 m) $\rightarrow$ VM(1, 5 m) $\rightarrow$ RH
A5	A / Explicit	“Search for cylinders and map every one you find, then return.”	None — both cylinders fully mapped; only RH remains. Tests false-replan rate.	NOM
B1	B / Implicit	“Search for cylinders and map every one you find, then return.”	None — search complete; cylinder manifest is empty; remaining plan still has approach+map steps.	REPLAN $\rightarrow$ RH
B2	B / Implicit	“Search for cylinders and map every one you find, then return.”	None — 3 cylinders in manifest; cylinder 1 mapped; remaining plan covers only cylinder 2 (cylinder 3 absent).	REPLAN $\rightarrow$ A(2) $\rightarrow$ M(2) $\rightarrow$ A(3) $\rightarrow$ M(3) $\rightarrow$ RH
B3	B / Implicit	“Do a thorough search, then map every cylinder you find twice.”	None — remaining plan uses “all” placeholder; cylinders 1 and 2 now specifically known.	REPLAN $\rightarrow$ A(1) $\rightarrow$ M(1) $\times$ 2 $\rightarrow$ A(2) $\rightarrow$ M(2) $\times$ 2 $\rightarrow$ RH
B4	B / Implicit	“Find any cylinders and map each one 3 times.”	None — 3 orbits of cylinder 1 done; only RH remains. Tests false-replan rate.	NOM
C1	C / Explicit	“Search for cylinders and map every one you find, then return.”	“Battery telemetry reports 15% remaining.”	REPLAN $\rightarrow$ RH
C2	C / Explicit	“Search for cylinders and map every one you find, then return.”	“Battery telemetry reports 55% remaining.”	NOM
C3	C / Explicit	“Search for cylinders and map every one you find, then return.”	“Battery telemetry reports 20% remaining.” Two full approach+map pairs remain (cylinders 2 and 3).	REPLAN $\rightarrow$ RH

TABLE VIII

EXPERIMENT 3: MULTI-STAGE REPLANNING CHAINS. EACH ROW IS ONE CHECKPOINT WITHIN A CHAIN. EVENT NOTATION: ADV(n) = n STEPS ADVANCED TO HISTORY (SUCCESSFUL EXECUTION); FAIL = CURRENT STEP ABORTED AND POPPED FROM TAIL WITHOUT ENTERING HISTORY; INJ = INJECTCYLINDER AND/OR INJECTFAILURE; CLR = CLEARFAILURE. TIERS: SIMPLE (1 CYLINDER, 1 FAILURE); MEDIUM (2 CYLINDERS, 2–3 FAILURES); COMPLEX (VARIED FAILURE MODES); LONG (NO FAILURES, REGRESSION CHECK).

Chain	Tier	CP	Events	Failure Signal Provided to Replanner	Expected Outcome
S1	Simple	0	—	None. Plan correctly covers cylinder 1.	NOM
		1	Adv(1), Fail	“Map orbit of cylinder 1 aborted mid-arc: IMU dropout. Recorded point-cloud data is unusable.”	REPLAN → A(1) → M(1) → RH
		2	Clr, Adv(3)	None. All steps advanced cleanly.	NOM
S2	Simple	0	Adv(1), Inj(cyl4)	“New cylinder detected at approximately 14 m during return home.”	REPLAN → A(2) → M(2) → RH
		1	Clr, Adv(3)	None. All steps advanced cleanly.	NOM
S3	Simple	0	Inj(cyl2)	“High-confidence detection of an untracked cylinder at approximately 12 m.”	REPLAN → A(2) → M(2) → RH
		1	Clr	None. Replanner extended tail; failure cleared.	NOM
		2	Adv(2)	None. approach(2) and map(2) advanced; RH remains.	NOM
M1	Medium	0	—	None. Plan correctly covers both cylinders.	NOM
		1	Adv(3), Inj	“Visual contact with cylinder 2 lost after approach before planned mapping. Relative position to cylinder 2 is unreliable.”	REPLAN → A(2) → M(2) → RH
		2	Clr, Adv(2), Inj(cyl6)	“High-confidence detection of untracked cylinder at approximately 16 m.”	REPLAN → A(3) → M(3) → RH
		3	Clr, Adv(3)	None. All three cylinders handled; chain converged.	NOM
M2	Medium	0	—	None. Plan correctly covers 2 vertical laps.	NOM
		1	Adv(1), Fail	“Vertical map lap 1 of cylinder 1 aborted: unrecoverable IMU spike after first arc segment. Lap 1 data is unusable.”	REPLAN → A(1, 5 m) → VM(1, 5 m) → VM(1, 5 m) → RH
		2	Clr, Adv(2), Inj	“Cylinder 1 centroid drifted 0.4 m during lap 2 vertical sweep. Drift is within the accepted 0.6 m tolerance.”	NOM
		3	Clr, Adv(2)	None. Lap 2 and RH advanced cleanly.	NOM
M3	Medium	0	—	None. Remaining plan correctly covers cylinder 2.	NOM
		1	Adv(1), Fail	“Map orbit of cylinder 2 aborted: camera shutter failure. All frames corrupted. Cylinder 2 has 0 valid mapping passes.”	REPLAN → A(2) → M(2) → RH
		2	Clr, Fail	“Approach to cylinder 2 aborted: Standoff geometry was not established.”	REPLAN → A(2) → M(2) → RH
		3	Clr, Adv(3)	None. All steps advanced cleanly.	NOM
C1	Complex	0	—	None. Plan correctly covers cylinder 1.	NOM
		1	Adv(2), Inj(cyl11)	“High-confidence detection of untracked cylinder at approximately 11 m during return leg.”	REPLAN → A(2) → M(2) → RH
		2	Clr, Inj(cyl7)	“High-confidence detection of untracked cylinder at approximately 17 m during approach to cylinder 2.”	REPLAN → A(2) → M(2) → A(3) → M(3) → RH
		3	Clr, Adv(4)	None. All three cylinders handled; only RH remains.	NOM
C2	Complex	0	—	None. Plan correctly covers both cylinders.	NOM
		1	Adv(2), Fail	“Approach to cylinder 2 aborted: UAV entered no-fly zone boundary before reaching standoff. Standoff was never established.” Mission abort policy stated in original mission: if approach or mapping of any cylinder fails for any reason, skip that cylinder and return home immediately, do not retry.	REPLAN → RH
		2	Clr, Adv(1)	None. RH advanced; chain converged.	NOM
C3	Complex	0	—	None. No cylinders discovered; RH is correct.	NOM
		1	Inj(cyl7)	“Cylinder detected at approximately 7 m.” Mission specifies: orbit the first cylinder twice.	REPLAN → A(1) → M(1)×2 → RH

TABLE IX

EXPERIMENT 4: REPLANNING UNDER COMPOUND (SIMULTANEOUS) FAILURE EVENTS. EVERY SCENARIO PRESENTS 2+ CONCURRENT SIGNALS WITH NO EXPLICIT PRIORITY RULES IN THE SYSTEM PROMPT. CATEGORIES: *safety vs mission* — CRITICAL SAFETY SIGNAL MUST OVERRIDE MISSION GOALS; *hard vs soft* — AN UNRECOVERABLE HARD CONSTRAINT ARRIVES ALONGSIDE AN IRRELEVANT SOFT WARNING; *conflict and adapt* — TWO SIGNALS REQUIRE A PRINCIPLED PARTIAL-MISSION TRADE-OFF; *recovery failure* — A SECOND FAILURE ARRIVES ON A PLAN THAT IS ALREADY A RECOVERY SEQUENCE, REQUIRING ESCALATION.

ID	Category	Exact Mission Text	Concurrent Failure Signals Provided to Re-planner	Expected Outcome
SV1	safety vs mission	“Search for cylinders and map every one you find, then return.”	(1) “Battery telemetry reports 12% remaining — critically low.” (2) “Lost tracking on cylinder 2 during approach; no confirmed detection for 10 s.”	REPLAN → RH
SV2	safety vs mission	“Search for cylinders and map every one you find, then return.”	(1) “Battery at 11% — critically low.” (2) “New cylinder detected at 18 m during search. Added to manifest as cylinder 3.”	REPLAN → RH
SV3	safety vs mission	“Find cylinders and orbit each one twice, then return.”	(1) “Battery at 14% — critically low.” (2) “Wind speed 6.2 m/s — drone experiencing roll oscillations.”	REPLAN → RH
SV4	safety vs mission	“Search for cylinders and map every one you find, then return.”	(1) “Battery at 10% — critically low.” (2) “Cylinder 1 visually confirmed at 5 m standoff.”	REPLAN → RH
HS1	hard vs soft	“Search for cylinders and map every one you find, then return.”	(1) “Obstacle avoidance triggered: uncharted obstacle at 8.0 m on bearing to cylinder 2. Cylinder 2 is physically unreachable.” (2) “Visual odometry drift detected: positional uncertainty increasing.”	REPLAN → RH
HS2	hard vs soft	“Find cylinders and orbit each one twice, then return.”	(1) “Motor controller fault on motor 3 — reduced thrust available.” (2) “Wind gusts reaching 3.5 m/s (within operational tolerance).”	REPLAN → RH
HS3	hard vs soft	“Search for cylinders and map every one you find, then return.”	(1) “Cylinder 3 not found — target coordinates yield no detectable object.” (2) “Depth sensor noise ± 0.3 m on last cylinder 3 estimate.”	REPLAN → RH
CA1	conflict and adapt	“Search for cylinders and map every one you find, then return.”	(1) “Battery at 20% — low.” (2) “Cylinder 2 is within standoff range (8 m); Cylinder 3 is 50 m away from current position.”	REPLAN → A(2) → M(2) → RH
CA2	conflict and adapt	“Search for cylinders and map every one you find, then return.”	(1) “Battery at 28% — low.” (2) “Cylinder 1 is 5 m away from current position. Cylinder 2 is 9 m away from current position. Cylinder 3 is 55 m away from current position.” (Cylinder 1 already mapped.)	REPLAN → A(2) → M(2) → RH
CA3	conflict and adapt	“Search for cylinders and map every one you find, then return.”	(1) “Cylinder 2 is 40 m away from current position.” (2) “Cylinder 3 is 8 m away from current position.” (3) “Cylinder 1 is 5 m away from current position.” (4) “Battery at 22% — low.” Signals intentionally presented in non-informative order; cylinder 1 already mapped.	REPLAN → A(3) → M(3) → RH
RF1	recovery failure	“Search for cylinders and map every one you find, then return.”	(1) “Approach attempt 2 for cylinder 2 failed. Obstacle avoidance triggered — uncharted obstacle at 7.5 m; cylinder 2 is physically unreachable.” (2) “Battery at 35%.” Remaining plan is already a recovery retry sequence for cylinder 2.	REPLAN → RH
RF2	recovery failure	“Find cylinders and orbit each one twice, then return.”	(1) “Battery at 18% — low.” (2) “Cylinder 1 orbit attempt 2 failed due to sensor timeout.” Remaining plan is already a recovery sequence covering both the cylinder 1 retry and cylinder 2.	REPLAN → RH